Mini Apps on Telegram

Fonte: https://core.telegram.org/api/bots/webapps · Extraído em 18 de outubro de 2026

Interactive HTML5 Mini Apps on Telegram can completely replace **any website**.

They support seamless authorization, integrated payments via multiple payment providers (with *Google Pay* and *Apple Pay* out of the box), delivering tailored push notifications to users, and much more.

This article offers a client-side overview of the implementation of bot mini apps using the MTProto API: see here for an overview of the mini-app side JS API ».

Main Mini Apps

Schema:

```
user#31774388 flags:# self:flags.10?true contact:flags.11?true
mutual_contact:flags.12?true deleted:flags.13?true bot:flags.14?true
bot_chat_history:flags.15?true bot_nochats:flags.16?true verified:flags.17?true
restricted:flags.18?true min:flags.20?true bot_inline_geo:flags.21?true
support:flags.23?true scam:flags.24?true apply_min_photo:flags.25?true
fake:flags.26?true bot_attach_menu:flags.27?true premium:flags.28?true
attach_menu_enabled:flags.29?true flags2:# bot_can_edit:flags2.1?true
close_friend:flags2.2?true stories_hidden:flags2.3?true
stories_unavailable:flags2.4?true contact_require_premium:flags2.10?true
bot_business:flags2.11?true bot_has_main_app:flags2.13?true
bot_forum_view:flags2.16?true bot_forum_can_manage_topics:flags2.17?true
bot_can_manage_bots:flags2.18?true bot_guestchat:flags2.19?true id:long
access_hash:flags.0?long first_name:flags.1?string last_name:flags.2?string
username:flags.3?string phone:flags.4?string photo:flags.5?UserProfilePhoto
status:flags.6?UserStatus bot_info_version:flags.14?int
restriction_reason:flags.18?Vector<RestrictionReason>
bot_inline_placeholder:flags.19?string lang_code:flags.22?string
emoji_status:flags.30?EmojiStatus usernames:flags2.0?Vector<Username>
stories_max_id:flags2.5?RecentStory color:flags2.8?PeerColor
profile_color:flags2.9?PeerColor bot_active_users:flags2.12?int
bot_verification_icon:flags2.14?long send_paid_messages_stars:flags2.15?long =
User;

webViewResultUrl#4d22ff98 flags:# fullsize:flags.1?true fullscreen:flags.2?true
query_id:flags.0?long url:string = WebViewResult;

botAppSettings#c99b1950 flags:# placeholder_path:flags.0?bytes
background_color:flags.1?int background_dark_color:flags.2?int
```

```
header_color:flags.3?int header_dark_color:flags.4?int = BotAppSettings;
botInfo#4d8a0299 flags:# has_preview_medias:flags.6?true user_id:flags.0?long
description:flags.1?string description_photo:flags.4?Photo
description_document:flags.5?Document commands:flags.2?Vector<BotCommand>
menu_button:flags.3?BotMenuButton privacy_policy_url:flags.7?string
app_settings:flags.8?BotAppSettings verifier_settings:flags.9?
BotVerifierSettings = BotInfo;

---functions---

messages.requestMainWebView#c9e01e7b flags:# compact:flags.7?true
fullscreen:flags.8?true peer:InputPeer bot:InputUser start_param:flags.1?string
theme_params:flags.0?DataJSON platform:string = WebViewResult;
```

Main Mini Apps are configured through @botfather.

Once enabled, the user.`bot_has_main_app` flag will be set, and an "Open App" button should be shown on the bot's profile page.

Clicking on this button should open the Main Mini App, by invoking messages.requestMainWebView: no URL needs to be passed to the method, because the Main Mini App URL is configured through @botfather.

Apps may also specify a custom background, header color and placeholder SVG logo to be used during loading screens, specified in the botAppSettings constructor, contained in the botInfo constructor returned in userFull.

After invoking messages.requestMainWebView and obtaining a webViewResultUrl result, clients should open a webview using the `url` contained in the returned webViewResultUrl.

The bot's profile page should also show a list of photos and videos, previewing the features offered by the Main Mini App, see Main Mini App previews for more info on how to configure and render them.

Main Mini Apps are also featured in the in-app Mini App Store ».

The Main Mini App should also be directly opened when clicking on a Main Mini App deep link »; the `compact` / `fullscreen` flag of the method must be set if the `mode` parameter in the link is set and equal to `compact` / `fullscreen`; any eventual `start_param` present in the link must also be passed to the method.

Main Mini App previews

```
botInfo#4d8a0299 flags:# has_preview_medias:flags.6?true user_id:flags.0?long
description:flags.1?string description_photo:flags.4?Photo
description_document:flags.5?Document commands:flags.2?Vector<BotCommand>
menu_button:flags.3?BotMenuButton privacy_policy_url:flags.7?string
app_settings:flags.8?BotAppSettings verifier_settings:flags.9?
BotVerifierSettings = BotInfo;
```

```
user#31774388 flags:# self:flags.10?true contact:flags.11?true
mutual_contact:flags.12?true deleted:flags.13?true bot:flags.14?true
bot_chat_history:flags.15?true bot_nochats:flags.16?true verified:flags.17?true
restricted:flags.18?true min:flags.20?true bot_inline_geo:flags.21?true
support:flags.23?true scam:flags.24?true apply_min_photo:flags.25?true
fake:flags.26?true bot_attach_menu:flags.27?true premium:flags.28?true
attach_menu_enabled:flags.29?true flags2:# bot_can_edit:flags2.1?true
close_friend:flags2.2?true stories_hidden:flags2.3?true
stories_unavailable:flags2.4?true contact_require_premium:flags2.10?true
bot_business:flags2.11?true bot_has_main_app:flags2.13?true
bot_forum_view:flags2.16?true bot_forum_can_manage_topics:flags2.17?true
bot_can_manage_bots:flags2.18?true bot_guestchat:flags2.19?true id:long
access_hash:flags.0?long first_name:flags.1?string last_name:flags.2?string
username:flags.3?string phone:flags.4?string photo:flags.5?UserProfilePhoto
status:flags.6?UserStatus bot_info_version:flags.14?int
restriction_reason:flags.18?Vector<RestrictionReason>
bot_inline_placeholder:flags.19?string lang_code:flags.22?string
emoji_status:flags.30?EmojiStatus usernames:flags2.0?Vector<Username>
stories_max_id:flags2.5?RecentStory color:flags2.8?PeerColor
profile_color:flags2.9?PeerColor bot_active_users:flags2.12?int
bot_verification_icon:flags2.14?long send_paid_messages_stars:flags2.15?long =
User;

botPreviewMedia#23e91ba3 date:int media:MessageMedia = BotPreviewMedia;
bots.previewInfo#0ca71d64 media:Vector<BotPreviewMedia>
lang_codes:Vector<string> = bots.PreviewInfo;

---functions---

bots.getPreviewMedias#a2a5594d bot:InputUser = Vector<BotPreviewMedia>;

messages.uploadMedia#14967978 flags:# business_connection_id:flags.0?string
peer:InputPeer media:InputMedia = MessageMedia;

bots.getPreviewInfo#423ab3ad bot:InputUser lang_code:string = bots.PreviewInfo;
bots.addPreviewMedia#17aeb75a bot:InputUser lang_code:string media:InputMedia =
BotPreviewMedia;
bots.editPreviewMedia#8525606f bot:InputUser lang_code:string media:InputMedia
new_media:InputMedia = BotPreviewMedia;
bots.deletePreviewMedia#2d0135b3 bot:InputUser lang_code:string
media:Vector<InputMedia> = Bool;
bots.reorderPreviewMedias#b627f3aa bot:InputUser lang_code:string
order:Vector<InputMedia> = Bool;
```

After enabling a main mini app » in @botfather, bots gain the ability to display **localized** preview medias (photos and videos) on their profile page, offering examples of what the app can do.

If a bot has some preview medias, the botInfo.`has_preview_medias` flag will be set (botInfo is contained in the bot's userFull).
Clients should then invoke bots.getPreviewMedias to fetch&download the preview medias once the user opens the bot's profile page.
The method will automatically select the correctly localized variant of each preview, according to the language code of the user (as passed to initConnection when first setting up the client).

Bot owners may edit the preview medias directly through the API.

To check whether the current user can edit the preview medias of a bot, make sure both the `bot_can_edit` and `bot_has_main_app` flags of the bot's user constructor are set.

Then, bots.getPreviewInfo should be invoked by bot owners to fetch the previously configured preview medias.
Pass an empty string to `lang_code` when first invoking the method to fetch the previously configured default preview medias (used as fallback if there is no preview for the current user's language), and the list of `lang_codes` for which there are localized previews; then re-invoke the method to fetch the previously configured previews for each of the returned `lang_codes`.

(Note: technically non-owners may also invoke bots.getPreviewInfo, but it will always behave exactly as bots.getPreviewMedias, returning only previews for the current language and an empty `lang_codes` array, regardless of the passed `lang_code`, so please only use bots.getPreviewMedias if you're not the owner).

Then, use bots.addPreviewMedia, bots.editPreviewMedia, bots.reorderPreviewMedias, bots.deletePreviewMedia, uploading medias using messages.uploadMedia as usual to setup fallback and localized previews for existing and new languages.

As specified above, each language can have a distinct set of previews that may be edited and reordered independently. The default, fallback language has an empty `lang_code`.

A maximum of bot_preview_medias_max » preview medias may be added per localization key.

Keyboard Button Mini Apps

Schema:

```
replyKeyboardMarkup#85dd99d1 flags:# resize:flags.0?true single_use:flags.1?
true selective:flags.2?true persistent:flags.4?true
rows:Vector<KeyboardButtonRow> placeholder:flags.3?string = ReplyMarkup;

keyboardButtonSimpleWebView#e15c4370 flags:# style:flags.10?KeyboardButtonStyle
text:string url:string = KeyboardButton;

messageActionWebViewDataSentMe#47dd8079 text:string data:string =
MessageAction;
messageActionWebViewDataSent#b4c38cb5 text:string = MessageAction;

webViewResultUrl#4d22ff98 flags:# fullsize:flags.1?true fullscreen:flags.2?true
```

```
query_id:flags.0?long url:string = WebViewResult;

---functions---

messages.requestSimpleWebView#413a3e73 flags:# from_switch_webview:flags.1?true
from_side_menu:flags.2?true compact:flags.7?true fullscreen:flags.8?true
bot:InputUser url:flags.3?string start_param:flags.4?string
theme_params:flags.0?DataJSON platform:string = WebViewResult;

messages.sendWebViewData#dc0242c8 bot:InputUser random_id:long
button_text:string data:string = Updates;
```

Keyboard Button Mini Apps should be opened when the user clicks a keyboardButtonSimpleWebView contained in a reply keyboard identified by a replyKeyboardMarkup constructor, by invoking messages.requestSimpleWebView passing the button's `url` to the `url` parameter.

After invoking messages.requestSimpleWebView and obtaining a webViewResultUrl result, clients should open a webview using the `url` contained in the returned webViewResultUrl.

Keyboard Button Mini Apps can send data back to the bot through the MTProto API via a `web_app_data_send` JS event ».

Upon receiving a `web_app_data_send` JS event » **only** from Keyboard Button Mini Apps, clients should invoke messages.sendWebViewData, passing the following arguments:

- `bot` - Bot ID
- `random_id` - Unique random ID to avoid resending the same event multiple times
- `button_text` - Text of the keyboardButtonSimpleWebView that was pressed to open the simple Mini App
- `data` - Contents of the `data` field of the JS event.

Make sure to ignore all `web_app_data_send` events sent after the first one, messages.sendWebViewData must be called only once. The webview must be closed after invoking the messages.sendWebViewData method.

This will generate a messageActionWebViewDataSent update for the user, and a messageActionWebViewDataSentMe update for the bot, containing the event data.

Inline button Mini Apps

Schema:

```
keyboardButtonWebView#e846b1a0 flags:# style:flags.10?KeyboardButtonStyle
text:string url:string = KeyboardButton;

webViewResultUrl#4d22ff98 flags:# fullsize:flags.1?true fullscreen:flags.2?true
query_id:flags.0?long url:string = WebViewResult;
```

```
inputBotInlineResult#88bf9319 flags:# id:string type:string title:flags.1?
string description:flags.2?string url:flags.3?string thumb:flags.4?
InputWebDocument content:flags.5?InputWebDocument
send_message:InputBotInlineMessage = InputBotInlineResult;
inputBotInlineResultPhoto#a8d864a7 id:string type:string photo:InputPhoto
send_message:InputBotInlineMessage = InputBotInlineResult;
inputBotInlineResultDocument#fff8fdc4 flags:# id:string type:string
title:flags.1?string description:flags.2?string document:InputDocument
send_message:InputBotInlineMessage = InputBotInlineResult;
inputBotInlineResultGame#4fa417f2 id:string short_name:string
send_message:InputBotInlineMessage = InputBotInlineResult;

updateWebViewResultSent#1592b79d query_id:long = Update;
webViewMessageSent#0c94511c flags:# msg_id:flags.0?InputBotInlineMessageID =
WebViewMessageSent;

---functions---

messages.requestWebView#269dc2c1 flags:# from_bot_menu:flags.4?true
silent:flags.5?true compact:flags.7?true fullscreen:flags.8?true peer:InputPeer
bot:InputUser url:flags.1?string start_param:flags.3?string
theme_params:flags.2?DataJSON platform:string reply_to:flags.0?InputReplyTo
send_as:flags.13?InputPeer = WebViewResult;

messages.prolongWebView#b0d81a83 flags:# silent:flags.5?true peer:InputPeer
bot:InputUser query_id:long reply_to:flags.0?InputReplyTo send_as:flags.13?
InputPeer = Bool;

messages.sendWebViewResultMessage#0a4314f5 bot_query_id:string
result:InputBotInlineResult = WebViewMessageSent;
```

Inline Button Mini Apps work similarly to inline bots »: they send messages on behalf of the user to the chat from which the query originated.

When the user clicks on an keyboardButtonWebView inline button contained in an inline keyboard identified by a replyInlineMarkup constructor, messages.requestWebView should be invoked, passing keyboardButtonWebView. `url` must be passed to the method's `url` parameter.

Then, clients should open a webview using the `url` contained in the returned webViewResultUrl.

After loading the webview, until it is closed by a web_app_close event, the user client must invoke messages.prolongWebView every 60 seconds: if the method call returns `QUERY_ID_INVALID`, the webview must be closed.

The opened URL's fragment parameters already contain basic information about the user and a `query_id` parameter, that is exposed by the bot Mini Apps JS library: this `query_id` can then be passed to the bot (within the Mini App itself, for example via an AJAX query or form submission to the server hosting the Mini App and the bot) and then used **by the bot** to invoke

messages.sendWebViewResultMessage, passing an InputBotInlineResult constructor that will automatically send a message with optionally attached media, and even inline buttons on behalf of the user.

Menu button Mini Apps

Menu button Mini Apps work similarly to inline button Mini Apps »: they send messages on behalf of the user to the chat from where the bot menu button » was clicked.

Menu button Mini Apps can be opened from a botMenuButton menu button »: in this case, the messages.requestWebView. `from_bot_menu` flag should be set, and the botMenuButton. `url` field must be passed to the method's `url` parameter.

The full flow is identical to the flow for inline button Mini Apps », apart from the different flags passed to messages.requestWebView, as described above.

Attachment menu Mini Apps

Attachment menu Mini Apps work similarly to inline button Mini Apps »: they send messages on behalf of the user to the chat where the bot's attachment menu » was opened.

Attachment menu Mini Apps can be opened from an attachment menu entry »: in this case, no special flag should be set when invoking messages.requestWebView.

Attachment menu Mini Apps can also be opened from a bot attachment menu deep link, in which case the `start_parameter` should be provided to messages.requestWebView. `start_param`, if present, and the `compact` / `fullscreen` flag must be set if the `mode` parameter is set and equal to `compact` / `fullscreen`.

The full flow is identical to the flow for inline button Mini Apps », apart from the different flags passed to messages.requestWebView, as described above.

Inline mode Mini Apps

```
messages.botResults#e021f2f6 flags:# gallery:flags.0?true query_id:long
next_offset:flags.1?string switch_pm:flags.2?InlineBotSwitchPM
switch_webview:flags.3?InlineBotWebView results:Vector<BotInlineResult>
cache_time:int users:Vector<User> = messages.BotResults;

inlineBotWebView#b57295d5 text:string url:string = InlineBotWebView;

webViewResultUrl#4d22ff98 flags:# fullsize:flags.1?true fullscreen:flags.2?true
query_id:flags.0?long url:string = WebViewResult;

---functions---

messages.getInlineBotResults#514e999d flags:# bot:InputUser peer:InputPeer
geo_point:flags.0?InputGeoPoint query:string offset:string =
```

```
messages.BotResults;

messages.requestSimpleWebView#413a3e73 flags:# from_switch_webview:flags.1?true
from_side_menu:flags.2?true compact:flags.7?true fullscreen:flags.8?true
bot:InputUser url:flags.3?string start_param:flags.4?string
theme_params:flags.0?DataJSON platform:string = WebViewResult;
```

Not to be confused with inline button mini apps ».

Inline mode Mini Apps can be used to generate a custom set of inline results in response to a user's inline query » via a `web_app_switch_inline_query` JS event ».

Inline mode Mini Apps can be opened by clicking on an inlineBotWebView button returned at the top of the inline result list, contained in messages.botResults. `switch_webview`, returned by messages.getInlineBotResults.

Pass the `url` to messages.requestSimpleWebView, while also setting the `from_switch_webview` flag.

After invoking messages.requestSimpleWebView and obtaining a webViewResultUrl result, clients should open a webview using the `url` contained in the returned webViewResultUrl.

Once the user has finished making their choices in the Mini App, a `web_app_switch_inline_query` JS event » should be emitted, containing a JSON object with the following fields:

- `query` - The inline query that will be inserted in the chat's input field, after the bot's username.
  May be an empty string, in which case just the bot's username will be inserted, triggering an empty inline query.
- `chat_types` - An array of strings, containing a combination of `users`, `bots`, `groups`, `channels`.
  If non-empty, the client should prompt the user to choose a specific chat of the specified type(s), then open the chosen chat and inserts the bot's username and the specified inline query in the input field.
  The array values specify which types of chats the user will be able to choose from.
  If empty, the current chat is used.

Upon receiving a `web_app_switch_inline_query` JS event » from the Mini App, the client should make a new inline query » to the same bot, with the newly specified `query`, either in the current chat or in the newly chosen chat, as specified by the `chat_types` field.

Side menu Mini Apps

```
webViewResultUrl#4d22ff98 flags:# fullsize:flags.1?true fullscreen:flags.2?true
query_id:flags.0?long url:string = WebViewResult;

---functions---
```

```
messages.requestSimpleWebView#413a3e73 flags:# from_switch_webview:flags.1?true
from_side_menu:flags.2?true compact:flags.7?true fullscreen:flags.8?true
bot:InputUser url:flags.3?string start_param:flags.4?string
theme_params:flags.0?DataJSON platform:string = WebViewResult;
```

Side menu Mini Apps can be opened by clicking on the installed side menu entry ».

This action must trigger a messages.requestSimpleWebView query with the `from_side_menu` flag set: clients should open a webview using the `url` contained in the returned webViewResultUrl.

After invoking messages.requestSimpleWebView and obtaining a webViewResultUrl result, clients should open a webview using the `url` contained in the returned webViewResultUrl.

Direct Link Mini Apps

Schema:

```
inputBotAppID#a920bd7a id:long access_hash:long = InputBotApp;
inputBotAppShortName#908c0407 bot_id:InputUser short_name:string = InputBotApp;

botAppNotModified#5da674b7 = BotApp;
botApp#95fcd1d6 flags:# id:long access_hash:long short_name:string title:string
description:string photo:Photo document:flags.0?Document hash:long = BotApp;

messages.botApp#eb50adf5 flags:# inactive:flags.0?true
request_write_access:flags.1?true has_settings:flags.2?true app:BotApp =
messages.BotApp;

webViewResultUrl#4d22ff98 flags:# fullsize:flags.1?true fullscreen:flags.2?true
query_id:flags.0?long url:string = WebViewResult;

---functions---

messages.getBotApp#34fdc5c3 app:InputBotApp hash:long = messages.BotApp;

messages.requestAppWebView#53618bce flags:# write_allowed:flags.0?true
compact:flags.7?true fullscreen:flags.8?true peer:InputPeer app:InputBotApp
start_param:flags.1?string theme_params:flags.2?DataJSON platform:string =
WebViewResult;
```

Another way to open Mini Apps is by using Direct Mini App links ».

These links are different from all other Mini App links, because they don't require the user to install an attachment menu, and a single bot can offer multiple Mini Apps, distinguished by their `short_name`.

These links should be handled as follows:

- Check if the `bot_username` parameter of the link is indeed a bot username, if so then
- Invoke messages.getBotApp, passing an inputBotAppShortName with the `short_name` contained in the `appname` query string parameter. If the client has already encountered an app with this short name from the same bot before, pass the `hash` of the cached botApp constructor to messages.getBotApp.
- If a messages.botApp constructor is returned, open the Mini App by invoking messages.requestAppWebView, generating an inputBotAppID constructor from `id` and `access_hash` of the returned botApp, or from previously cached information if we already met the bot app and botAppNotModified was returned.
    - If the client has clicked on the link in a Telegram chat, pass the chat's peer information into `peer`; otherwise pass the bot's peer information, instead.
    - If the messages.botApp. `request_write_access` flag is set, the bot is asking permission to send messages to the user: **ask confirmation from the user** with a prompt and a checkbox, and if the user agrees, set the `write_allowed` flag when invoking messages.requestAppWebView.
    - If the messages.botApp. `inactive` flag is set, **ask confirmation from the user** before opening the Mini App; the `request_write_access` checkbox should be shown in this prompt, if needed.
      Confirmation should **always** be asked, even if the `inactive` flag is not set, when opening the link from places where the full link is not visible (i.e. messageEntityTextUrl text links, inline buttons etc.).

      Don't use two prompts if confirmation to open the Mini App is required *and* `request_write_access` is set, use just one prompt with an optional checkbox for `request_write_access`.
      Bots whose IDs are contained in the `whitelisted_bots` » configuration parameter are exempt from this open-confirmation prompt.
    - If the `startapp` query string parameter is present, pass it to `start_param` when invoking messages.requestAppWebView.

Finally, open the webview using the `url` contained in the returned webViewResultUrl.

Outgoing events: Mini App to client

Mini Apps can *send* web events starting with `web_app_`; see the web event documentation for the full list of events that can be *sent* by the Mini App to the client ».

Incoming events: Client to Mini App

Mini Apps can also *receive* events, by exposing a `window.Telegram.WebView.receiveEvent("event_name", params)` method.

Here's the full list of events that can be *received* by a Mini App from the client, if the client invokes the `receiveEvent` method.

`main_button_pressed`

Params: `null`

Sent by the client when the user presses the main button located at the bottom of the webview, handle this event only if the main button was previously configured by a `web_app_setup_main_button` event ».

`back_button_pressed`

Params: `null`

Sent by the client when the user presses the (OS or UI) back button, if it was previously enabled by a `web_app_setup_back_button` event ».

`settings_button_pressed`

Params: `null`

Sent by the client when the user presses the settings button, if it was previously enabled by a `web_app_setup_settings_button` event ».

`invoice_closed`

Params: JSON object with the following fields:

- `slug` - Invoice identifier (string)
- `status` - One of the following values (string):
  - `cancelled` – The user closed the invoice popup without paying, before the call to payments.sendPaymentForm.
  - `failed` – The user tried to pay, but the payment failed: the call to payments.sendPaymentForm returned an RPC error and the popup was closed.
  - `pending` – The payment is still processing: the bot will receive a further service message about a successful payment. payments.sendPaymentForm was successfully invoked returning payments.paymentVerificationNeeded, the user completed all additional verification forms returned by the method and the invoice popup was closed, but the client hasn't received a messageActionPaymentSent service message yet.
    Note that eventual errors will not be sent as a `failed` event if the user fails additional validation (i.e. 3-D Secure) returned by payments.paymentVerificationNeeded: the state will remain `pending`.
  - `paid` – The invoice was paid successfully: the client completed the payment flow », the invoice popup was closed and a messageActionPaymentSent service message was received by the client.

Sent by the client to report the payment status of an invoice obtained from a `web_app_open_invoice` event ».

`viewport_changed`

Params: a JSON object with the following fields:

- `height` - The current height of the visible area of the Mini App (excluding the bottom main button, if visible) (integer)
- `is_state_stable` - If true, the viewport is currently being resized (animation in progress), more events of this type may be emitted. (boolean)
- `is_expanded` - Whether the Mini App is expanded to its maximum height after the user swiped up or after the Mini App emitted a web_app_expand event (boolean)

Emitted when the viewport is changed.

`theme_changed`

Params: a JSON object with the following fields:

- `theme_params` - A theme parameters object » (object)

Emitted when requested by the Mini App using a `web_app_request_theme` event », or when the app theme changes.

Theme parameters

Bot Mini Apps can be themed according to the following theme parameters, passed as a JSON object to the `theme_params` parameter of the messages.requestSimpleWebView, messages.requestWebView and messages.requestAppWebView methods.

This JSON object has the following keys, containing color theme information (hex string, RGB, no alpha) to pass to the Mini App.

See here » for more info on the contents of the object.

`popup_closed`

Params: a JSON object with an optional `button_id` string field.

Emitted when the user presses a button or cancels a popup brought up by a previous `web_app_open_popup` event ».

`write_access_requested`

Params: a JSON object with the following fields:

- `status` - `allowed` or `cancelled`

Used by clients to reply to a `web_app_request_write_access` event », indicating whether the user has allowed the bot to send messages to the user (`allowed`) or not (`cancelled`).

`phone_requested`

Params: a JSON object with the following fields:

- `status` - `sent` or `cancelled`

Used by clients to reply to a `web_app_request_phone` event », indicating whether the user has shared their phone number with the bot (`allowed`) or not (`cancelled`).

`biometry_info_received`

Params: a JSON object with the following fields:

- `available` - boolean, if true, indicates that biometric authentication is available on the current device.
- `type` - optional string, set if `available` is true, contains the type of biometric authentication, one of:
  - `finger` - fingerprint-based biometrics
  - `face` - face-based biometrics
  - `unknown` - biometrics of an unknown type
- `access_requested` - boolean, indicates whether the app has previously requested permission to use biometrics through a `web_app_biometry_request_access` event »
- `access_granted` - boolean, indicates whether the user has granted the app permission to use biometrics in response to a `web_app_biometry_request_access` event ».
  If false and `access_requested` is true, the user has denied the app permission to use biometrics, in which case the app should open a prompt notifying the user that the biometric settings must be changed to use biometrics, and if the user clicks on the in-app confirm button, a web_app_biometry_open_settings event » must be emitted.
- `token_saved` - boolean, whether a token was safely stored on-device by a previous web_app_biometry_update_token event ».
- `device_id` - string, a unique device identifier that can be used to match the token to the device.

Used by clients to reply to a `web_app_biometry_get_info` event » or a `web_app_biometry_request_access` event ».

`biometry_token_updated`

Params: a JSON object with the following fields:

- `status` - string, one of:
  - `updated` - If the token was successfully updated.
  - `removed` - If the token was successfully removed.
  - `failed` - If biometric authentication failed, or the app doesn't have permission to use biometrics (a biometry_info_received event » event will also be emitted if the app hasn't previously initialized the state using `web_app_biometry_get_info` event » or a `web_app_biometry_request_access` event »).

Used by clients to reply to a `web_app_biometry_update_token` event ».

`biometry_auth_requested`

Params: a JSON object with the following fields:

- `status` - string, either `authorized` or `failed`.
  If `failed`, a biometry_info_received event » event will also be emitted if the app hasn't previously initialized the state using `web_app_biometry_get_info` event » or a `web_app_biometry_request_access` event ».
- `token` - optional string, set if `status` is `authorized`, contains the token previously set using the `web_app_biometry_update_token` event ».

Used by clients to reply to a `web_app_biometry_request_auth` biometric authentication request ».

`custom_method_invoked`

Params: a JSON object with the following fields:

- `req_id` - The `req_id` from the `web_app_invoke_custom_method` request
- `result` - The JSON data contained in the response of the bots.invokeWebViewCustomMethod method, if the method call succeeded
- `error` - The text of the RPC error, if the method call failed

Used by clients to reply to `web_app_invoke_custom_method` events ».

`clipboard_text_received`

Params: a JSON object with the following fields:

- `req_id` - The `req_id` from the `web_app_read_text_from_clipboard` request
- `data` - A string with the clipboard contents (optional, if not provided consider the request failed)

Used by clients to reply to `web_app_read_text_from_clipboard` events ».

`qr_text_received`

Params: a JSON object with the following fields:

- `data` - string with the contents of a scanned QR code.

Emitted by clients if a new QR code was scanned by the native QR code scanner opened with a `web_app_open_scan_qr_popup` event ».

`scan_qr_popup_closed`

Params: `null` or an empty object

Emitted by clients if the QR code scanner popup opened with a `web_app_open_scan_qr_popup` event » was closed by the user or failed to open altogether due to permission issues.

`visibility_changed`

Params: a JSON object with a single `is_visible=true|false` boolean field.

Emitted the Mini App becomes active (`true`) or inactive (`false`), e.g., opened/closed from minimized state or selected/deselected among tabs).

`secondary_button_pressed`

Params: `null`

Sent by the client when the user presses the secondary button located at the bottom of the webview, handle this event only if the secondary button was previously configured by a `web_app_setup_secondary_button` event ».

`fullscreen_changed`

Params: a JSON object with the following fields:

- `is_fullscreen` - boolean indicating whether the mini app is currently in fullscreen mode.
- `blur_enabled` - Optional boolean, present when `is_fullscreen=true`, indicating whether background content is blurred in fullscreen mode.

Sent by the client when replying to web_app_request_fullscreen and web_app_exit_fullscreen events.

`fullscreen_failed`

Params: a JSON object with the following fields:

- `error` - string indicating the error, one of:
  - `UNSUPPORTED` - Fullscreen mode is not supported on this device or platform
  - `ALREADY_FULLSCREEN` - The Mini App is already in fullscreen mode

Sent by the client if a failure occurs while handling web_app_request_fullscreen and web_app_exit_fullscreen events.

`accelerometer_started`

Params: `null`

Sent by the client in response to a web_app_start_accelerometer event » if accelerometer tracking is started successfully.

Until web_app_stop_accelerometer is emitted by the mini app, the client will also emit accelerometer_changed events at most every `refresh_rate` milliseconds with accelerometer readings.

`accelerometer_failed`

Params: JSON object with the following fields:

- `error` - String, one of:
  - `UNSUPPORTED` - Accelerometer tracking is not supported on this device or platform.

Sent by the client in response to a web_app_start_accelerometer event » if accelerometer tracking cannot be started.

`accelerometer_stopped`

Params: `null`

Sent by the client in response to a web_app_stop_accelerometer event » if accelerometer tracking is stopped successfully.

`accelerometer_changed`

Params: JSON object with `x`, `y` and `z` fields of type `float`, containing the current acceleration in the X, Y and Z-axis, measured in m/s².

Sent periodically by the client at most every `refresh_rate` milliseconds after web_app_start_accelerometer » is invoked and until web_app_stop_accelerometer is invoked.

`gyroscope_started`

Params: `null`

Sent by the client in response to a web_app_start_gyroscope event » if gyroscope tracking is started successfully.

Until web_app_stop_gyroscope is emitted by the mini app, the client will also emit gyroscope_changed events at most every `refresh_rate` milliseconds with gyroscope readings.

`gyroscope_failed`

Params: JSON object with the following fields:

- `error` - String, one of:
  - `UNSUPPORTED` - Gyroscope tracking is not supported on this device or platform.

Sent by the client in response to a web_app_start_gyroscope event » if gyroscope tracking cannot be started.

`gyroscope_stopped`

Params: `null`

Sent by the client in response to a web_app_stop_gyroscope event » if gyroscope tracking is stopped successfully.

`gyroscope_changed`

Params: JSON object with `x`, `y` and `z` fields of type `float`, containing the current rotation rate around the X, Y and Z-axis, measured in rad/s.

Sent periodically by the client at most every `refresh_rate` milliseconds after web_app_start_gyroscope » is invoked and until web_app_stop_gyroscope is invoked.

`device_orientation_started`

Params: `null`

Sent by the client in response to a web_app_start_device_orientation event » if device orientation tracking is started successfully.

Until web_app_stop_device_orientation is emitted by the mini app, the client will also emit device_orientation events at most every `refresh_rate` milliseconds with device orientation readings.

`device_orientation_failed`

Params: JSON object with the following fields:

- `error` - String, one of:
  - `UNSUPPORTED` - Device orientation tracking is not supported on this device or platform.

Sent by the client in response to a web_app_start_device_orientation event » if device orientation tracking cannot be started.

`device_orientation_stopped`

Params: `null`

Sent by the client in response to a web_app_stop_device_orientation event » if device orientation tracking is stopped successfully.

`device_orientation_changed`

Params: JSON object with the following fields:

- `alpha` - The rotation around the Z-axis, measured in radians.
- `beta` - The rotation around the X-axis, measured in radians.
- `gamma` - The rotation around the Y-axis, measured in radians.
- `absolute` - A boolean that indicates whether or not the device is providing orientation data in absolute values (may be false even if absolute data was requested if and only if the device/platform doesn't support absolute orientation tracking).

Sent periodically by the client at most every `refresh_rate` milliseconds after web_app_start_device_orientation » is invoked and until web_app_stop_device_orientation is invoked.

`home_screen_added`

Params: `null`

Sent by the client in response to a web_app_add_to_home_screen event » if the shortcut was (already) added successfully.

It is acceptable to not emit this event if the current platform doesn't have a way to determine the installation status of the shortcut.

`home_screen_failed`

Params: JSON object with the following fields:

- `error` - String, one of:
  - `UNSUPPORTED` - Shortcuts are not supported on this platform, or installation of the shortcut failed, or status cannot be reported.

Sent by the client in response to a web_app_add_to_home_screen event » if the shortcut could not be added.

It is acceptable to not emit this event if the current platform doesn't have a way to determine the installation status of the shortcut.

`home_screen_checked`

Params: JSON object with the following fields:

- `status` - String, one of:
  - `unsupported` – the feature is not supported, and it is not possible to add the icon to the home screen,
  - `unknown` – the feature is supported, and the icon can be added, but it is not possible to determine if the icon has already been added,
  - `added` – the icon has already been added to the home screen,
  - `missed` – the icon has not been added to the home screen.

Always sent by the client in response to a web_app_check_home_screen event ».

`emoji_status_failed`

Params: JSON object with the following fields:

- `error` - String, one of:
  - `UNSUPPORTED` – The feature is not supported by the client.
  - `SUGGESTED_EMOJI_INVALID` – One or more emoji identifiers are invalid.
  - `DURATION_INVALID` – The specified duration is invalid.
  - `USER_DECLINED` – The user closed the dialog without setting a status.
  - `SERVER_ERROR` – A server error occurred when attempting to set the status.
  - `UNKNOWN_ERROR` – An unknown error occurred.

Sent by the client in response to a web_app_set_emoji_status event » if the emoji status could not be set.

`emoji_status_set`

Params: `null`

Sent by the client in response to a web_app_set_emoji_status event » if the emoji status was set successfully.

`emoji_status_access_requested`

Params: JSON object with the following fields:

- `status` - String, one of:
  - `allowed` – The user (already) granted the bot permission to edit their emoji status
  - `cancelled` – The user declined the request, or an error occurred.

Sent by the client in response to a web_app_request_emoji_status_access event ».

`file_download_requested`

Params: a JSON object with the following fields:

- `status` - Either `cancelled` (the download was aborted by the user or was not allowed by the API) or `downloading` (the download has successfully started).

Emitted by clients during the mini app file download flow, initiated by the web_app_request_file_download » event.

`prepared_message_failed`

Params: a JSON object with the following fields:

- `error` - Contains the text of the RPC error returned by messages.getPreparedInlineMessage, or `USER_DECLINED` if the user aborted the sharing process, or `MESSAGE_SEND_FAILED` if sending fails for another reason.

Emitted by clients in case of errors during the share prepared messages flow, initiated by the web_app_send_prepared_message » event.

`prepared_message_sent`

Params: `null`

Emitted by clients when the share prepared messages flow initiated by the web_app_send_prepared_message » event completes successfully.

`safe_area_changed`

Params: a JSON object with the following fields:

- `top` - The top inset in pixels, representing the space to avoid at the top of the screen (integer).
- `bottom` - The bottom inset in pixels, representing the space to avoid at the bottom of the screen (integer).
- `left` - The left inset in pixels, representing the space to avoid on the left side of the screen (integer).
- `right` - The right inset in pixels, representing the space to avoid on the left side of the screen (integer).

Emitted by clients when any of the system-defined safe area inset padding values change, or when explicitly requested by the mini app using web_app_request_safe_area.

See here » for more info.

`content_safe_area_changed`

Params: a JSON object with the following fields:

- `top` - The top inset in pixels, representing the space to avoid at the top of the content area (integer).
- `bottom` - The bottom inset in pixels, representing the space to avoid at the bottom of the content area (integer).
- `left` - The left inset in pixels, representing the space to avoid on the left side of the content area (integer).
- `right` - The right inset in pixels, representing the space to avoid on the left side of the content area (integer).

Emitted by clients when any of the content-defined safe area inset padding values change, or when explicitly requested by the mini app using web_app_request_content_safe_area.

See here » for more info.

`location_requested`

Params: a JSON object with the following fields:

- `available` - Boolean, indicates whether location data is available or not; if false, the following fields must not be set.
- `latitude` - Float, contains latitude in degrees.
- `longitude` - Float, contains longitude in degrees.
- `altitude` - Float or null, contains the altitude above the sea level in meters, must be null if not supported by the device or platform.
- `course` - Float or null, contains the direction the device is moving in degrees, must be null if not supported by the device or platform.
- `speed` - Float or null, contains the speed of the device in m/s, must be null if not supported by the device or platform.
- `horizontal_accuracy` - Float or null, contains the accuracy of the latitude and longitude values in meters, must be null if not supported by the device or platform.

- `vertical_accuracy` - Float or null, contains the accuracy of the altitude value in meters, must be null if not supported by the device or platform.
- `course_accuracy` - Float or null, contains the accuracy of the course value in degrees, must be null if not supported by the device or platform.
- `speed_accuracy` - Float or null, contains the accuracy of the speed value in m/s, must be null if not supported by the device or platform.

Emitted in response to a web_app_request_location event.

`location_checked`

Params: a JSON object with the following fields:

- `available` - Boolean, indicates whether location data is available on the current platform or device.
- `access_requested` - Optional boolean, must be set if `available` is true, indicates whether the mini app has already requested access to location data.
- `access_granted` - Optional boolean, if set and `true` indicates that the user granted the mini app access to location data.

Emitted in response to a web_app_check_location event.

`device_storage_key_saved`

Params: a JSON object with the following fields:

- `req_id` - String, `req_id` from the requesting web_app_device_storage_save_key event

Indicates the key was saved or removed successfully.

Emitted in response to a web_app_device_storage_save_key event.

`device_storage_key_received`

Params: a JSON object with the following fields:

- `req_id` - String, `req_id` from the requesting web_app_device_storage_get_key event
- `value` - String or null, the requested value or no value if the specified key is not set.

Returns the requested key.

Emitted in response to a web_app_device_storage_get_key event.

`device_storage_cleared`

Params: a JSON object with the following fields:

- `req_id` - String, `req_id` from the requesting web_app_device_storage_clear event

Indicates the storage was cleared.

Emitted in response to a web_app_device_storage_clear event.

`device_storage_failed`

Params: a JSON object with the following fields:

- `req_id` - String, `req_id` from the requesting event
- `error` - String, the error that prevented the request from succeeding.

Emitted in response to a web_app_device_storage_save_key, web_app_device_storage_get_key or web_app_device_storage_clear event.

`secure_storage_key_saved`

Params: a JSON object with the following fields:

- `req_id` - String, `req_id` from the requesting web_app_secure_storage_save_key event

Indicates the key was saved or removed successfully.

Emitted in response to a web_app_secure_storage_save_key event.

`secure_storage_key_received`

Params: a JSON object with the following fields:

- `req_id` - String, `req_id` from the requesting web_app_secure_storage_get_key event
- `value` - String or null, the requested value or no value if the specified key is not set.
- `can_restore` - Boolean, true if `value` is null but the value can be restored using a web_app_secure_storage_restore_key event

Returns the requested key.

Emitted in response to a web_app_secure_storage_get_key event.

`secure_storage_key_restored`

Params: a JSON object with the following fields:

- `req_id` - String, `req_id` from the requesting web_app_secure_storage_restore_key event
- `value` - String, the restored value

Restores the requested key.

Emitted in response to a web_app_secure_storage_restore_key event.

`secure_storage_cleared`

Params: a JSON object with the following fields:

- `req_id` - String, `req_id` from the requesting web_app_secure_storage_clear event

Indicates the storage was cleared.

Emitted in response to a web_app_secure_storage_clear event.

`secure_storage_failed`

Params: a JSON object with the following fields:

- `req_id` - String, `req_id` from the requesting event
- `error` - String, the error that prevented the request from succeeding; most notably, `UNSUPPORTED` for platforms that do not support secure storage.

Emitted in response to a web_app_secure_storage_save_key, web_app_secure_storage_get_key, web_app_secure_storage_restore_key or web_app_secure_storage_clear event.

`oauth_supported`

Params: a JSON object with the following fields:

- `version` - Integer, currently must be equal to `1`

Indicates that the current client supports OAuth login ».

Unconditionally emitted in response to all oauth_request events ».

`oauth_result_confirmed`

Params: a JSON object with the following fields:

- `result_url` - String or null, contains the final redirection URL

Emitted in response to a non-empty and valid oauth_request event » if the OAuth login flow » succeeded.

`oauth_result_failed`

Params: Null or an empty JSON object.

Emitted in response to a non-empty and valid oauth_request event » if the OAuth login flow » failed or if it was explicitly cancelled by the user.

`requested_chat_sent`

Params: a JSON object with the following fields:

- `req_id` - String, `req_id` from the requesting web_app_request_chat event

Indicates that the requested peers were sent to the bot using messages.sendBotRequestedPeer.

Emitted in response to a web_app_request_chat event, see here » for the full flow.

`requested_chat_failed`

Params: a JSON object with the following fields:

- `req_id` - String, `req_id` from the requesting web_app_request_chat event

- `error` - Optional string, the error that prevented the request from succeeding.

Indicates that the peer request was cancelled or has failed.

Emitted in response to a web_app_request_chat event, see here » for the full flow.